

Domain 2

Lesson 1

Python v2

Branching Statements: if

Conditions are important parts of decisions and loops that control the flow of a program. An if statement performs an action if a condition is true. For example, an if statement might print a particular message if a person's score in a game is over 10,000.

Purpose

Upon completing this project, you will better understand the purpose of if statements and how to use them in code.

Steps for Completion

1. Review the code, then answer the questions about the if statement.

```
1  score = 12000
2
3  if score > 10000:
4      print("You have reached level 2")
5  print("Thank you for playing")
```

- a. What will be the output of running this code?
 - i. the first print statement and the second
- b. What would the output be if the score on line 1 were changed to 5000?
 - i. only the second print statement

Project Details

Project file

N/A

Estimated completion time

5 minutes

Video reference

Domain 2

Topic: Branching Statements

Subtopic: if

Objectives covered

2 Flow Control with Decisions and Loops

2.1 Construct and analyze code segments that use branching statements

2.1.1 if

Branching Statements: elif

An elif (else if) statement is used when a code segment needs to check for multiple possibilities for conditions and then perform an action based on which condition in a set of conditions is true.

Purpose

Upon completing this project, you will better understand the purpose of elif statements and how to use them to build code.

Steps for Completion

1. Review the code, then answer the questions about the if and elif statements.

```

1  score = 8000
2
3  if score > 10000:
4      print("You have reached level 2")
5  elif score > 5000:
6      print("You have reached level 1")
7

```

- a. What will the output of running this code be?
 - i. You have reached level 1
- b. What will the output be if the score on line 1 is changed to 15000?
 - ii. You have reached level 2

Project Details

Project file

N/A

Estimated completion time

5 minutes

Video reference

Domain 2

Topic: Branching Statements

Subtopic: elif

Objectives covered

2 Flow Control with Decisions and Loops

2.1 Construct and analyze code segments that use branching statements

2.1.2 elif

Purpose

Steps for Completion

- ```
1 score = 3000
2
3 if score > 10000:
4 print("You have reached level 2")
5 elif score > 5000:
6 print("You have reached level 1")
7 else:
8 print("You need to score higher to reach a level")
9
10 print("Thank you for playing")
```

- i. You need a higher score to reach a level

- j. You have reached level 1

## Project file

### Estimated completion time

## Video reference

**Topic:** Branching Statements

## Objectives covered

## 2.1 Construct and analyze code segments that use branching statements

### 2.1.3 else

# Nested/Compound Conditions

Often, more complicated statements are needed to create more complex code. For example, in a nested if statement, one if statement goes inside of another and is used if a condition requires checking another. On the other hand, a compound conditional statement contains multiple conditions within the same statement. Programmers might use this type of statement when one of two conditions must be true for a specific code to run.

## Purpose

Upon completing this project, you will better understand nested and compound conditions and how to use them within code.

## Steps for Completion

1. Open the **214-nested.py** file from your Domain 2 Student folder.
2. What type of statement is on line 10 of the code in this file?
  - a. elif
3. Are the coins listed on line 1 immutable?
  - a. no
4. Run the code. What is the output?
  - a. You reached level 1. Keep Going

### Project Details

#### Project file

214-nested.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Branching Statements

**Subtopic:** Nested and Compound Conditions

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.1** Construct and analyze code segments that use branching statements

**2.1.4** Nested and compound conditional expressions

# Review 2.1

## Purpose

Upon completing this project, you will be better equipped to work through logic in code using decisions and loops and be able to determine why a block of code is not producing the desired result.

## Steps for Completion

1. Open the **214-analyze.py** file from your Domain 2 Student folder.
2. The anticipation for this code is that it returns the print statement on line 7, as a player with a gold coin who has reached 10000 points should be moved to level 3. Right now, it is returning the print statement on line 13. What two problems with the code are causing this issue?
  - a. gold is not capitalized at the g  
 \_\_\_\_\_  
 \_\_\_\_\_
  - b. The if scord > 10000 is not if score >= 10000  
 \_\_\_\_\_  
 \_\_\_\_\_
3. Return to the video to compare your answers to the video's answers.

### Project Details

#### Project file

214-analyze.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Branching Statements

**Subtopic:** Nested and Compound Conditions; Review 2.1

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.1** Construct and analyze code segments that use branching statements



# Domain 2

# Lesson 2

Python v2

# Iteration: while

Iterations, also known as loops, perform an action repeatedly when run. A while loop runs while conditions are true or false. For example, a while loop might run until the score in a game reaches 100 points.

## Purpose

Upon completing this project, you will better understand the purpose of while loops and how to use them in code.

## Steps for Completion

1. Open the **221-while.py** file from your Domain 2 Student folder.
2. What would happen if the score increase on line 2 was changed to 0?
  - a. it runs infinit amounts of times
3. Edit the code so that the output will print a message for having 100 points. Run the code to check your work.
4. Save the file as **221-while-completed**

### Project Details

#### Project file

221-while.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Iteration

**Subtopic:** while

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.2** Construct and analyze code segments that perform iteration

**2.2.1** while



# Iteration: for

A for loop runs for a predetermined number of iterations based on the length of a collection. As an example, a for loop might iterate over a list or a number of positions.

## Purpose

Upon completing this project, you will better understand the purpose of for loops and how to use them in code.

## Steps for Completion

1. Open the **222-for.py** file from your Domain 2 Student folder.
2. Run the code. What is the starting number for the positions in the output?
  - a. 1
3. Edit the range so that the numbers for the positions in the output will print as positions 1 through 10. Run the code to check your work.
3. Add a comma and the number **2** at the end of the range and run the code. How does this change the output?
  - a. counted in steps of 2 so 1, 3, 5, 7, 9
4. Save the file as **222-for-completed**

### Project Details

#### Project file

222-for.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Iteration

**Subtopic:** for

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.2** Construct and analyze code segments that perform iteration

**2.2.2** for

The break keyword is used to exit a loop once a desired point is reached. The continue keyword skips an iteration of a loop.

## Purpose

Upon completing this project, you will better understand the purpose of the break and continue keywords and how to use them in code.

## Steps for Completion

1. Review the code, then answer the question about the break keyword.

```
1 coins = ('Bronze', 'Silver', 'Platinum', 'Gold')
2 for coin in coins:
3 print('You possess a', coin, 'coin.')
4 if coin == 'Platinum':
5 print('Congratulations! You move to the next level!')
6 break
```

- a. What is the result of the break keyword in this code?

- i. to exit the loop

2. Open the **224-continue.py** file from your Domain 2 Student folder.
3. Edit the code so that the "You possess a Platinum coin" statement does not print as part of the output if the coin is platinum.
4. Save the file as **224-continue-completed**

## Project Details

## Project file

224-continue.py

### Estimated completion time

5 minutes

## Video reference

## Domain 2

**Topic:** Iteration

**Subtopic:** break; continue

## Objectives covered

## 2 Flow Control with Decisions and Loops

## 2.2 Construct and analyze code segments that perform iteration

### 2.2.3 break

## 2.2.4 continue

# Domain 2 Lesson 3

Python v2

# Iteration: pass

The pass keyword allows one to build parts of code while waiting for information. For example, a message as part of a loop may be unknown, but the loop still needs to be tested. The pass keyword can make the code-writing process more efficient by allowing developers to work on other parts of the code, even if the code currently in the file is unfinished.

## Purpose

Upon completing this project, you will better understand the purpose of the pass keyword.

## Steps for Completion

1. The pass keyword functions as a(n) placeholder for code that will be added later.
2. Open the **225-pass.py** file from your Domain 2 Student folder.
3. Edit the code so a placeholder replaces the print statement on line 5.
4. Save the file as **225-pass-completed**

### Project Details

#### Project file

225-pass.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Iteration

**Subtopic:** pass

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.2** Construct and analyze code segments that perform iteration

**2.2.5** pass

# Nested Loops

While loops are used for conditional situations, and for loops are run a set number of times. Developers may sometimes want one loop inside another, known as a nested loop. A nested loop allows developers to run a process inside of another process.

## Purpose

Upon completing this project, you will better understand how to use nested loops.

## Steps for Completion

1. Open the **226-nested.py** file from your Domain 2 Student folder.
2. Edit the code so the while statement runs on the third life and the first two lives.
3. Edit the code so the positions shown when the code runs are 1 through 10.
4. Save the file as **226-nested-completed**

### Project Details

#### Project file

226-nested.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Iteration

**Subtopic:** Nested Loops

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.2** Construct and analyze code segments that perform iteration

**2.2.6** Nested loops



# Loops with Compound Conditions

A compound conditional statement is a statement with two or more conditions. Using a compound conditional operator saves time as it prevents one from having to run multiple if statements to get the same results from code.

## Purpose

Upon completing this project, you will better understand how to use compound conditional statements.

## Steps for Completion

1. Open the **227-compound.py** file from your Domain 2 Student folder.
2. Edit the code so the if statement on line 5 is a compound conditional statement requiring the scepter variable to be True.
3. Save the file as **227-compound-completed**

## Project Details

### Project file

227-compound.py

### Estimated completion time

5 minutes

### Video reference

#### Domain 2

**Topic:** Iteration

**Subtopic:** Loops with Compound Conditions

### Objectives covered

**2** Flow Control with Decisions and Loops

**2.2** Construct and analyze code segments that perform iteration

**2.2.7** Loops that include compound conditional expressions

# Review 2.2

## Purpose

Upon completing this project, you will be able to analyze code with loops better and deduce the number of times a loop will run.

## Steps for Completion

1. Open the **227-analyze.py** file from your Domain 2 Student folder.
2. How many print statements will run with the code in its current state?
  - a. \_\_\_\_\_
3. Change the continue statement to a break statement.
4. How many print statements will run with the code in its new state?
  - a. \_\_\_\_\_
5. Return to the video to get the answers to the questions in this review exercise.

### Project Details

#### Project file

227-analyze.py

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Iteration

**Subtopic:** Loops with Compound Conditions; Review 2.2

#### Objectives covered

**2** Flow Control with Decisions and Loops

**2.2** Construct and analyze code segments that perform iteration